Explanation Using Defeasible Rule sets

Augustine Mochoeneng*‡
MCHKOK003@myuct.ac.za
University of Cape Town
Cape Town, Western Cape, South Africa

Abstract

Symbolic classifiers that reason by default, retracting a conclusion only when a more specific exception applies, offer predictions that are interpretable by construction rather than through post-hoc justification. Exception Closed Conjunctive Rule Sets (ECCRS) realize this idea directly: classification is decided by a Most Specific Wins operator, so the explanation for any prediction is simply the chain of rules and exceptions that produced it. This structural account is easy to read, but it does not on its own show that the decision is justified under an established theory of reasoning with exceptions.

We build a workbench that checks ECCRS's decisions against such a theory. Building on a correspondence established by Madzime and Meyer, it translates an ECCRS rule set into the equivalent form used in the KLM framework, a well-established formal account of how conclusions should be drawn and withdrawn in the presence of exceptions, and tests whether one of its standard reasoning methods, Lexicographic Closure, reaches the same verdicts as ECCRS. On rule sets meeting the conditions the translation requires, the two agree on every tested instance. We then run the workbench [1] on rule sets that break those conditions, and find that disagreement arises only when a condition is violated, and never as a wrong label: the formal reasoner withholds a decision or reports a contradiction rather than contradicting ECCRS's answer. Comparing the two sides' explanations, we show that the formal account can support and supplement ECCRS's own rule-based justifications.

1 Introduction

Systems utilizing Machine Learning models are increasingly deployed across domains involving high-stakes decisions, where they replace, improve, or assist decisions traditionally made by humans. However, their predictions often rely on internal representations, weights, embeddings, and learned feature interactions that have no inherent correspondence to human reasoning. This raises an important question: if users do not trust a model or its predictions, will they use it [25]? And if machines can learn representations of physical systems that outperform human models, can we, in turn, learn from the learners [28] ? These questions motivate explainable artificial intelligence (XAI), a suite of methods intended to make the internal logic of ML systems interpretable to human reasoning. [2] [27] [28]

Any decision-making system, not only those built on machine learning, faces an explainability problem when its decisions are used to justify consequential outcomes. Arrieta et al. define explainability as the property by which a model can produce details or reasons that make its functioning easy and clear to understand [2]. Without this property, an unjustified or incorrect decision cannot be properly examined or corrected if its reasoning is inaccessible. Explanations are central to scientific inquiry, connecting abstract models to causal understanding and experimental validation [28]. This has motivated post-hoc XAI methods such as feature attribution [1] and counterfactual explanations [23], which attempt to reconstruct human-understandable reasoning after a decision has been made. However, a post-hoc method cannot be completely faithful to the model it describes; if it were, it would simply be the model itself.

Symbolic rule-based classifiers mitigate the problem of explainability by construction rather than reconstruction. They consist of explicit "IF-THEN" decision rules over meaningful features [11] [24], making their reasoning interpretable by design [27]. However, not all rule-based formats provide equally reliable explanations. Decision Lists [26], for example, resolve decisions according to the first rule that matches an input, making their conclusions dependent on rule ordering [11]. Decision Sets instead collect all applicable rules and rely on an external tie-breaking mechanism to determine the final class [15]. Both approaches therefore introduce aspects of the decision process that are external to the rules themselves.

We study Exception Closed Conjunctive Rule Sets (ECCRS) [20], a classifier designed for this setting of rule sets with structurally encoded exceptions. Similar to Decision Sets, ECCRS collects the rules applicable to an input, and a decision class is then determined by the Most Specific Wins (MSW) operator, which resolves conflicts among applicable rules entirely through a structural specificity relationship encoded in the rule bodies themselves. Because this resolution requires no external machinery, ECCRS produces explanation traces directly from its own inference process. Making it not only interpretable but also explainable [19].

ECCRS' Most specific rule decision determines which applicable rule wins, but it does not, on its own, establish that the resulting decision is semantically justified under a formal theory of reasoning with exceptions. This motivates checking ECCRS against such a theory: the KLM framework, a standard formal account of defeasible reasoning, where conclusions hold by default but may be withdrawn when more specific information applies, developed by Kraus, Lehmann and Magidor [14]. The framework offers several concrete methods for deciding what a set of default rules implies; we use one of the best known, Lexicographic Closure.

Bridging the two requires a translation from an ECCRS rule set to the form KLM reasoning operates on, a defeasible knowledge base. A prior result, which we call the alignment theorem, guarantees that under three structural conditions on the rule set, ECCRS's decisions and Lexicographic Closure's entailments coincide on every input. The payoff of establishing such a correspondence is practical, not only foundational. If a classifier's decisions can be checked against a formal entailment relation, the check becomes a tool: it can audit a learned rule set, flagging the structural defects that would make

*

[1] Source code available at https://github.com/auggieToo/ECCRS-ENGINE

its defeasible reading unsound, and it fails safely, since a reasoner that abstains or reports a contradiction under a malformed rule set is preferable to a classifier that silently returns a wrong label. This turns the verification from a one-off validation into a consistency check that a rule learner could run as a matter of course. This project has two aims. The first is to verify the correspondence in practice: to check, on rule sets learned from real data, whether ECCRS's Most Specific Wins decisions and Lexicographic Closure's entailments coincide, and to do so under a precise notion of agreement. For each instance we compute both verdicts and assign the pair to one of five outcomes, agreement, joint abstention, one-sided abstention, conflict (the reasoner entails both labels), or disagreement (the two commit to opposite labels). The alignment theorem predicts that, on rule sets meeting its conditions, every covered instance, one with at least one applicable rule, falls into the first two categories; a single disagreement or conflict on such a rule set would show the theorem's practical relevance to be limited. On rule sets satisfying the conditions, we find agreement on every covered instance.

The second aim is to characterise what happens when those conditions fail. Here we state a hypothesis: breaking a structural assumption can cost a decision its decisiveness or its consistency, but cannot flip its label. That is, a malformed rule set may lead the reasoner to withhold a decision, or to signal a contradiction, but never to confidently assert the opposite of what the classifier decided. If this holds, the verification layer fails only in safe ways, never as a silent wrong answer. Our violation experiments test it by breaking each assumption in turn; we explain in Section 7 why the translation guarantees it.

We make three contributions. First, we implement the ECCRS-to-KLM translation of Madzime and Meyer [20] as an executable workbench that, given an exported rule set and a set of instances, computes for each both the MSW decision and the Lexicographic Closure entailment. Second, we use it to confirm their correspondence empirically on benchmark rule sets, and to compare the explanation traces the two sides produce. Third, we test the label-preservation hypothesis by constructing rule sets that violate each assumption in turn, characterising the failure each produces and complementing the theoretical failure analysis of [19, 20] with a per-assumption empirical account.

The remainder of the paper is organised as follows. Sections 2 and 3 introduce ECCRS, MSW, and defeasible reasoning. Section 4 outlines the work that has been done in this field. Section 5 describes the workbench and translation process and then presents the evaluation methodology, followed by the results in Section 6. Section 7 discusses the implications and limitations of the findings, and Section 8 concludes the paper.

2 Exception-Closed Conjunctive Rule Sets

Consider a bank's automated loan approval system. The bank has developed a lending policy governing when applicants should be approved or denied loans based on their financial profile. The relevant features are defined in Table 1.

The initial policy is simple. Stably employed applicants who meet income requirements are approved, but if they have a prior default they are denied. Good credit alone is sufficient for approval, but if they have a fraud flag they are denied. Madzime et al. [21] introduced the Defeasible Horn Classifier with Exceptions (DHCE), which handles this policy well. DHCE is a rule-based classifier that makes default–exceptions explicit by attaching each exception directly to its parent default rule. This applies the "normally…unless…" principle, making decisions easier to trace without using any post-hoc explanations. The DHCE rule set for this policy might look as in Table 2.

Table 1: Bank loan policy features.

Feature	Meaning
Stable	Applicant has stable employment
Income	Applicant meets minimum income threshold
Credit	Applicant has good credit history
Default	Applicant has a prior loan default
Fraud	Applicant has a fraud flag on record

Table 2: The loan approval rules.

Rule	Condition	Decision
r1	stable ∧ income	approve
r2	stable ∧ income ∧ default	deny
r3	credit	approve
r4	credit ∧ fraud	deny

The classifier is tested with the following applicants.

(1) Auggie, who has stable employment and the minimum income required (stable ∧ income), is approved for a loan. *(r1 applies.)*
(2) Dani has good credit and a fraud flag (credit ∧ fraud) and is denied a loan. *(r3 fires but r4 overrides.)*

Each trace is short, complete, and faithful. However, DHCE imposes two constraints that limit its formal scope.

First, exceptions are attached only one level deep; exceptions to exceptions of rules are not supported. This choice keeps every override local to its default and preserves a short, readable trace where each default appears next to its own exception [21]. Suppose the bank's risk team revise the policy. They observe that applicants with a prior default who can provide a secured asset against the loan represent an acceptable risk and should be approved. A new feature `secured` could then be introduced.

Rule	Condition	Decision
r5	stable ∧ income ∧ default ∧ secured	approve

This is an entirely reasonable policy revision. Rule r5 is an exception to r2, which is itself an exception to r1. The chain reads naturally: *"stable-income applicants are approved, unless they have a prior default, unless they can secure the loan."* Now consider a fourth applicant, David.

(3) David has stable employment, meets income requirements, has a prior default, but has secured the loan.

The correct decision under the revised policy is *approve*: r5 overrides r2, which had overridden r1. This three-level chain cannot be represented in DHCE. Rule r5 would have to be encoded as a separate independent default, losing its structural connection to r1 and r2, and with it the readable exception narrative.

Secondly, DHCE still assigns numeric priorities externally to resolve conflicts between defaults, placing part of the resolution, in the language of Darwiche and Hirth [8], outside the complete reason for the decision. Both limitations share a root: conflict resolution in DHCE is only partially intrinsic. What is needed is a classifier in which it is fully intrinsic at every level, exception chains reach arbitrary depth, and predictions are certifiable by an independent reasoner. Exception-Closed Conjunctive Rule Sets (ECCRS) are designed to meet these requirements.

ECCRS operates on Boolean features. Each instance is represented by a complete feature pattern, an assignment of exactly one signed literal, either a_i or $\neg a_i$, for every feature atom a_i in the domain. David's profile would be

$$F_{David} = \text{Stable} \wedge \text{Income} \wedge \neg\text{Credit} \wedge \text{Default} \wedge \text{Secured} \wedge \neg\text{Fraud}.$$

A rule is a pair (B, ℓ) where B is a *consistent conjunction* of signed literals (the rule body, encoding the condition under which the rule applies) and $\ell \in \{0, 1\}$ is the class label (the prediction the rule makes if the conditions are met). The rule policy translates directly:

$$\text{r1:}\quad \text{stable} \wedge \text{income} \Rightarrow 1 \qquad \text{r2:}\quad \text{stable} \wedge \text{income} \wedge \text{default} \Rightarrow 0$$

where label 1 means approve the loan and label 0 means deny the loan. A rule (B, ℓ) is *applicable* to an instance x if the instance satisfies every literal in the body B, that is $x \models B$. Intuitively, a rule applies when all of its conditions are met by the instance being classified.

Not every rule in a rule set is relevant to every instance. When classifying a loan applicant, only the rules whose conditions are actually satisfied by that applicant's profile should be considered. A rule that requires good credit history is simply irrelevant to an applicant who does not have good credit: it cannot fire, it cannot contribute to the prediction, and it should not appear in the explanation. The set of applicable rules for a given instance is the pool of rules that are genuinely in play for that classification decision.

Given a rule set $\mathcal{R}$ and a complete feature pattern F, a rule $(B, \ell) \in \mathcal{R}$ is *applicable* to F if F satisfies every literal in the rule body B, i.e. $F \models B$. The set of applicable rules for F is

$$\mathcal{A}(F) = \{(B, \ell) \in \mathcal{R} : F \models B\}.$$

For David, $\mathcal{A}(F_{David}) = \{r_1, r_2, r_5\}$. These rules are the candidates for determining David's classification.

Two rule bodies B and C are *comparable* if $B \subset C$ or $C \subset B$, one is a strict subset of the other, meaning one rule is strictly more specific. The bodies of r_1, r_2, and r_5 form a strict inclusion chain: $B_{r_1} \subset B_{r_2} \subset B_{r_5}$. This hierarchy is the structural relationship ECCRS uses to resolve conflicts, with no external priority needed. If two opposite-label rules are comparable, specificity determines the winner. If two opposite-label rules are compatible, in that they can both apply to the same instance, but not comparable, no structural basis for resolution exists and external tie-breaking machinery would be required, placing the resolution outside the complete reason for the decision [8].

Exception closure is the constraint that prevents this situation from arising. A rule set is *relaxed exception closed* on a training domain D if, for every pair of opposite-label rules (B, ℓ) and $(C, 1-\ell)$, comparability is only required when both rules co-apply on some instance of the training domain D. This is enforceable from D and gives no guarantees about unseen feature combinations: the Boolean feature space contains 2^n different patterns and the training data covers only a small fraction of these. A rule set is *strictly exception closed* if comparability is required for every compatible opposite-label pair, regardless of whether any training instance triggers both; this is enforced over the full Boolean feature space. Strict exception closure is the stronger requirement and is the one relevant to this project: the alignment between ECCRS and defeasible-entailment reasoning holds when the rule set is strictly exception closed.

2.1 Most Specific Wins

Given a complete feature pattern F, the Most Specific Wins principle identifies the inclusion-maximal applicable rules, those not strictly overridden by any more specific applicable rule:

$$\mathcal{M}(F) := \{(B, \ell) \in \mathcal{A}(F) : \nexists\, (C, \ell') \in \mathcal{A}(F) \text{ with } B \subset C\}.$$

The MSW prediction is then

$$\text{MSW}_{\mathcal{R}}(F) = \begin{cases} \ell & \text{if } \mathcal{M}(F) \neq \emptyset \text{ and all rules in } \mathcal{M}(F) \text{ have label } \ell, \\ \bot & \text{if } \mathcal{M}(F) = \emptyset \text{ or } \mathcal{M}(F) \text{ has rules with mixed labels} \end{cases}$$

Under strict exception closure, $\mathcal{M}(F)$ is guaranteed to contain only rules of the same label on every covered pattern; the mixed-label case cannot arise. The argument is direct. Suppose two rules (B, ℓ) and $(C, 1-\ell)$ both belong to $\mathcal{M}(F)$, both inclusion-maximal with opposite labels. They both apply to F, so $F \models B$ and $F \models C$, meaning B and C are compatible. By strict exception closure, compatible opposite-label rules must be comparable, either $B \subset C$ or $C \subset B$. But if $B \subset C$ then (B, ℓ) is overridden by $(C, 1-\ell)$ and cannot be in $\mathcal{M}(F)$, a contradiction; symmetrically for $C \subset B$. Therefore no two rules in $\mathcal{M}(F)$ can have opposite labels under strict exception closure. When no rule applies, MSW returns $\bot$, meaning the classifier abstains rather than forcing a prediction.

For David, the inclusion chain is $B_{r_1} \subset B_{r_2} \subset B_{r_5}$: r_1 is overridden by r_2, which in turn is overridden by r_5. Therefore $\mathcal{M}(F_{David}) = \{r_5\}$, giving the prediction approve.

ECCRS is thus able to reach conclusions that are traceable. Other, already-established frameworks would produce the same result as ECCRS, so a question one may ask is whether this newly established rule-set framework aligns with them in some way.

3 Defeasible Reasoning

The structural priority that governs ECCRS, where more specific rules override the conclusions of more general ones, mirrors a pattern of reasoning studied formally in the KLM framework of Kraus, Lehmann, and Magidor [14]. We assume familiarity with classical propositional logic, in which $\models$ denotes entailment; Ben-Ari [3] gives a full treatment. KLM extends propositional logic with defeasible conditionals of the form $\alpha \mathrel{|\!\sim} \beta$, read as "typically, if α then β," and characterises classes of nonmonotonic inference through rationality postulates and corresponding semantic models. Within this

framework, several entailment relations have been proposed; three are relevant here. Rational Closure (RC) [17] is the most conservative, endorsing the fewest conclusions and serving as a lower bound any reasonable defeasible reasoner should meet. Lexicographic Closure (LC) [16] and System W [13] both extend RC, drawing every conclusion it draws and more in cases where it is overly cautious, with LC preserving the maximum amount of knowledge when reasoning about exceptional cases [7]. All three derive their conclusions from the same computational foundation, a ranking of knowledge-base statements by exceptionality; what differs is how each uses that ranking to answer a query. This project's workbench is aligned against Lexicographic Closure, which therefore receives the fullest treatment below; Rational Closure is developed as the procedure LC builds on.

A defeasible knowledge base is a collection of defeasible conditionals encoding what typically holds in a domain. Before any entailment algorithm can rank or reason with these conditionals, it needs a way to identify which of them describe exceptional situations, those whose antecedents conflict with the typical behaviour described by the rest of the knowledge base. This identification is done through the *materialisation* of the knowledge base.

Formally, a defeasible knowledge base $\mathcal{K}$ is a finite set of defeasible conditionals. Its materialisation $\overrightarrow{\mathcal{K}}$ replaces every defeasible conditional with its classical material counterpart [12]:

$$\overrightarrow{\mathcal{K}} := \{\alpha \to \beta : \alpha \mathrel{|\!\sim} \beta \in \mathcal{K}\}.$$

A conditional $\alpha \mathrel{|\!\sim} \beta$ is *exceptional* with respect to $\mathcal{K}$ if $\overrightarrow{\mathcal{K}} \models \neg\alpha$, that is, if α conflicts with the typical behaviour of the full knowledge base [12]. The loan knowledge base is

$$\mathcal{K} = \begin{cases} \text{Stable} \wedge \text{Income} \mathrel{|\!\sim} \text{Approve}, \\ \text{Stable} \wedge \text{Income} \wedge \text{Default} \mathrel{|\!\sim} \neg\text{Approve}, \\ \dots \end{cases}$$

Under materialisation, rules 2, 3, and 5 are exceptional: they describe atypical situations that conflict with the typical behaviour of the knowledge base. Exceptionality is the concept all three entailment algorithms use to rank conditionals, and it is defined from the materialisation alone.

3.1 Rational Closure

Rational Closure ranks the conditionals in the knowledge base by how exceptional they are. When answering a query with an exceptional antecedent, it ignores the conditionals describing typical behaviour that the antecedent violates and reasons from a more specific knowledge base.

The BaseRank algorithm. RC uses BaseRank [7] to rank the conditionals by their degree of exceptionality. It proceeds iteratively: starting from the full materialisation $E_0 = \overrightarrow{\mathcal{K}}$, it repeatedly identifies which conditionals remain exceptional with respect to the current layer, peeling away the most general layer at each step until nothing changes. A conditional's rank is the last layer in which it appears before being removed; the higher the rank, the more exceptional the conditional.

Algorithm 1 BaseRank

Input: Knowledge base $\mathcal{K}$
Output: Ordered tuple $(R_0, \dots, R_{n-1}, R_\infty, n)$

$i := 0$
$E_0 := \overrightarrow{\mathcal{K}}$
repeat
  $E_{i+1} := \{\alpha \to \beta \in E_i \mid E_i \models \neg\alpha\}$
  $R_i := E_i \setminus E_{i+1}$
  $i := i + 1$
until $E_{i-1} = E_i$
$R_\infty := E_{i-1}$
if $E_{i-1} = \emptyset$ **then**
  $n := i - 1$
else
  $n := i$
end if
return $(R_0, \dots, R_{n-1}, R_\infty, n)$

The base rank of a conditional $\alpha \mathrel{|\!\sim} \beta$ is

$$\text{br}_{\mathcal{K}}(\alpha \mathrel{|\!\sim} \beta) := \begin{cases} i & \text{if } \alpha \to \beta \in E_i \setminus E_{i+1}, \\ \infty & \text{if } \alpha \to \beta \in E_n. \end{cases}$$

Conditionals assigned rank ∞ are maximally exceptional. Applying BaseRank to the first three rules of the loan knowledge base gives

$$\begin{aligned} E_0 &= \overrightarrow{\mathcal{K}}, \\ E_1 &= \{\text{Stable} \wedge \text{Income} \wedge \text{Default} \mathrel{|\!\sim} \neg\text{Approve}, \\ &\qquad \text{Stable} \wedge \text{Income} \wedge \text{Default} \wedge \text{Secured} \mathrel{|\!\sim} \text{Approve}\}, \\ E_2 &= \{\text{Stable} \wedge \text{Income} \wedge \text{Default} \wedge \text{Secured} \mathrel{|\!\sim} \text{Approve}\}. \end{aligned}$$

Crucially, this ranking reconstructs the same specificity ordering that ECCRS's MSW uses: more specific rule bodies correspond to higher-ranked conditionals.

The Rational Closure algorithm. Given the base ranking, a query $\alpha \mathrel{|\!\sim} \beta$ is answered as follows. If α is exceptional with respect to the current knowledge, the lowest-ranked conditionals are removed and the check is repeated; removal continues until α is no longer exceptional, at which point the remaining knowledge is tested for classical entailment of $\alpha \to \beta$. This mirrors a defeasible reasoner's response to an atypical situation: ignore the general defaults that do not apply, and reason from the specific knowledge that does.

RC is intentionally cautious. Because it discards an entire rank at a time, it can drop conditionals unrelated to the query. In the classic penguin example, on learning that penguins are exceptional birds RC discards all rank-0 conditionals, including "birds typically have wings," and so refuses to conclude that penguins have wings even though nothing contradicts it [16]. This bluntness is what Lexicographic Closure refines.

3.2 Lexicographic Closure

Lehmann [16] identified RC's rank-at-a-time discarding as a limitation: rather than dropping whole ranks, a reasoner should preserve as many conditionals as possible and discard only what is strictly necessary to make the exceptional antecedent consistent. Where

RC is maximally cautious about drawing conclusions, LC is maximally cautious about discarding knowledge [7]. Concretely, when a rank must be weakened, LC does not remove it wholesale; it removes the least-serious subset of that rank whose removal restores consistency, keeping the fewest and most general conditionals it can.

This refinement is realised by replacing the base ranking with a finer *subset ranking*. SubsetRank begins from the base ranking and, within each rank, splits it by subset size so that discarding fewer conditionals is preferred to discarding more.

Algorithm 2 SubsetRank

Input: Knowledge base $\mathcal{K}$
Output: Ordered tuple $(R_0, \ldots, R_k, R_\infty, k+1)$

$(B_0, \ldots, B_{m-1}, B_\infty, m) := \textsc{BaseRank}(\mathcal{K})$
$i := 0; \quad k := 0$
repeat
  for $j := |B_i|$ **to** 1 **do**
    $S_{i,j} := \textsc{Subsets}(B_i, j)$
    $D_{i,j} := \bigvee_{X \in S_{i,j}} \bigwedge_{x \in X} x$
    $R_k := D_{i,j}$
    $k := k + 1$
  end for
  $i := i + 1$
until $i = m$
$R_\infty := B_\infty$
return $(R_0, \ldots, R_k, R_\infty, k+1)$

Given the subset ranking, LC answers a query with exactly the RC procedure described above, with the single change that the ranking is produced by SubsetRank rather than BaseRank. Because SubsetRank refines each base rank into finer layers, the removal loop discards conditionals a subset at a time rather than a rank at a time, which is precisely the behaviour Lehmann's refinement calls for. On the penguin example, LC removes only the conditional in conflict and retains "birds typically have wings," so it concludes that penguins have wings, where RC does not.

3.3 Translating ECCRS into a KLM Knowledge Base

An ECCRS rule set classifies by structural specificity; a KLM entailment relation classifies by formal defeasible consequence. To compare the two, we translate a rule set $\mathcal{R}$ into a defeasible knowledge base $\mathcal{K}(\mathcal{R})$ over which entailment can be computed.

We introduce a single propositional atom m to denote the positive class: m stands for label 1 and $\neg m$ for label 0. Each ECCRS rule is mapped to one defeasible conditional according to its label,

$$(B, 1) \mapsto B \mathrel{|\!\sim} m, \qquad (B, 0) \mapsto B \mathrel{|\!\sim} \neg m,$$

so the knowledge base is

$$\mathcal{K}(\mathcal{R}) = \{B \mathrel{|\!\sim} m : (B, 1) \in \mathcal{R}\} \cup \{B \mathrel{|\!\sim} \neg m : (B, 0) \in \mathcal{R}\}.$$

Representing both classes with the single atom m, rather than two independent label atoms, makes the two possible decisions exact negations of one another: the query for label 0 is the query for $\neg m$, and the abstention case, in which neither m nor $\neg m$ is entailed, is expressed directly.

The translation is only useful if the entailments of the knowledge base track the decisions of the classifier. The alignment theorem states that, under three structural assumptions on the rule set, they coincide exactly on every complete feature pattern.

Assumptions. The result requires the exported rule set $\mathcal{R}$ to satisfy:

(1) **Strict global exception closure.** For every compatible opposite-label pair $(B, \ell), (C, 1-\ell) \in \mathcal{R}$, the bodies are comparable: $B \subset C$ or $C \subset B$. This rules out unstructured cross-label conflict anywhere in the feature space.
(2) **Sanity.** No two rules share an identical body with opposite labels.
(3) **No total override.** Every rule has at least one complete pattern on which it is not defeated by a stricter opposite-label rule; formally, for every $(B, \ell) \in \mathcal{R}$ there is a complete pattern F with $F \models B$ and, for all $(C, 1-\ell) \in \mathcal{R}$, $B \subset C \Rightarrow F \not\models C$.

Madzime and Meyer further show that only the first is independent: sanity follows from strict global exception closure, and no total override is restored by a prediction-preserving pruning of totally overridden rules that leaves every MSW decision unchanged [20]. This bears on our violation study. Because sanity follows from closure, a sanity violation is necessarily also a closure violation: two rules with identical bodies and opposite labels are compatible and neither strictly extends the other, which is exactly what closure forbids. We therefore do not present the three variants as breaking three logically independent assumptions; they break the structural conditions in three distinct ways, the sanity variant being the degenerate case of two identical opposite-label bodies, the closure variant the general case of two compatible but incomparable bodies, and the total override variant leaving closure intact but breaking the auxiliary no-total-override condition.

Theorem. Let $\mathcal{K} = \mathcal{K}(\mathcal{R})$ be the translated knowledge base of a rule set satisfying the three assumptions. Then for every complete feature pattern F:

(1) if no ECCRS rule applies to F, then $\mathcal{K}$ entails neither $F \mathrel{|\!\sim} m$ nor $F \mathrel{|\!\sim} \neg m$;
(2) if $\mathrm{MSW}_{\mathcal{R}}(F) = 1$, then $\mathcal{K}$ entails $F \mathrel{|\!\sim} m$ and does not entail $F \mathrel{|\!\sim} \neg m$;
(3) if $\mathrm{MSW}_{\mathcal{R}}(F) = 0$, then $\mathcal{K}$ entails $F \mathrel{|\!\sim} \neg m$ and does not entail $F \mathrel{|\!\sim} m$.

The classifier's three possible verdicts, abstain, class 1, and class 0, therefore correspond exactly to the three entailment outcomes on the translated knowledge base.

4 Related Works

The formal study of reasoning with exceptions has a long history, with default logic [4], circumscription [9], and preferential approaches all proposed to capture inferences that can be withdrawn in the light of more specific information. Among these, the preferential and rational semantics of Kraus, Lehmann, and Magidor [14] have become the dominant formal account, supplying both the rationality postulates that characterise well-behaved defeasible

entailment and the ranked-model semantics on which concrete entailment relations such as Rational and Lexicographic Closure are built.

A recurring theme in this literature is the transplantation of KLM semantics into other representation formalisms, so that a formalism gains a principled account of defeasible consequence without redeveloping one from scratch. Defeasible description logics extend the KLM programme to ontological knowledge, importing rational-closure-style reasoning into the description-logic setting [5], and related work carries the same machinery into standpoint logics [18] and rule languages such as disjunctive Datalog [22]. Closest to the present work, Carr et al. [6] give a defeasible reading of Formal Concept Analysis by mapping its implications onto KLM conditionals and studying the resulting entailment. In each case the move is the same: an existing symbolic formalism is translated into KLM semantics to inherit its formal guarantees. This is the pattern our ECCRS-to-KLM translation follows, and Carr et al. is its nearest structural precedent, translating a rule-like formalism into KLM and examining the correspondence.

A separate line of work uses KLM not as a target to translate into but as a system whose own conclusions are to be explained. Research on justification for defeasible entailment develops procedures and tooling that, given a conclusion drawn under an entailment relation such as Rational Closure, produce the reasons that support it [29]. This work has grown as explainability has become a stated goal for defeasible reasoners, and it is directly concerned with making KLM entailment transparent. It is important to be precise about what this achieves and what it does not: it explains why a conclusion follows within the KLM framework. It does not take a decision made by some other, independently defined system and check whether that decision agrees with what KLM would entail. The object being explained originates inside KLM; there is no external classifier whose behaviour is being validated.

This distinction marks the gap the present work addresses. Prior work either translates a formalism into KLM to study the resulting semantics, or explains conclusions already drawn inside KLM, but in neither case is KLM entailment used as an external standard against which a separately specified classifier's decisions are verified. We do exactly this: ECCRS makes its predictions by a structural, specificity-driven rule (Most Specific Wins) with no reference to KLM, and we then ask whether those predictions coincide, instance by instance, with the entailments of Lexicographic Closure on the translated knowledge base. The classifier and the entailment relation are defined independently, and the correspondence between them is the object of study rather than an assumed property, which distinguishes this work from both translation and justification lines above.

5 Materials and Methods

5.1 Design and Implementation

This section describes the design and implementation of the ECCRS-KLM alignment and explanation traces workbench. Given an ECCRS rule set and a set of instance features, the workbench first validates that the structural assumptions the alignment theorem relies on are satisfied, halting with a report if any of the assumptions are violated. On a compliant ECCRS rule set, the workbench continues along two paths: the ECCRS side runs MSW inference to classify each instance and produce explanation traces, while a translation module maps the same rule set into a KLM Knowledge Base, from which we compute BaseRank, SubsetRank and Lexicographic Closure algorithms. The two paths converge in an agreement check, comparing MSW's classification against KLM entailment for every instance. Figure 1 gives an overview of this pipeline; the remainder of the section details each stage.

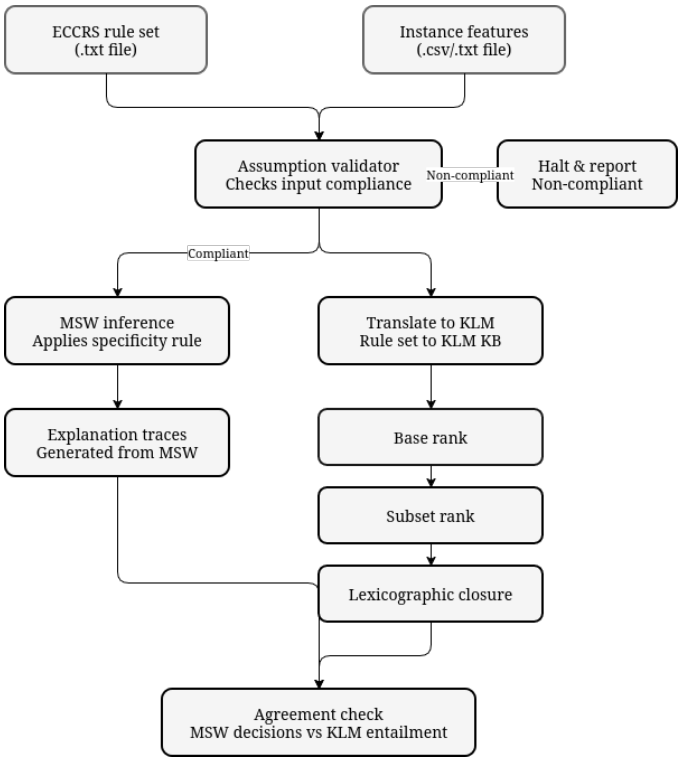


Figure 1: Overview of the ECCRS-KLM workbench pipeline.

The two paths are computed independently by design. Because the agreement check measures whether MSW's structural decision procedure coincides with KLM entailment, deriving one from the other would make agreement trivial; each path must reach its classification on its own for the comparison to be meaningful.

Input Format. The workbench accepts a numbered ECCRS rule set as a `.txt` file. An example of an input rule is:

$$2 : \textbf{IF}\ a(1){=}0 \wedge a(2){=}1 \wedge a(5){=}0\ \ \textbf{THEN}\ 1$$

The number preceding each rule serves as its unique identifier and is referenced when generating the explanation traces for the MSW inference. The notation $a(i)$ denotes the i-th feature of the dataset from which the rule set was learned, while the value following THEN specifies the decision class assigned to the rule.

Assumption Validation. The three assumptions, *Sanity*, *Strict Global Exception Closure*, and *No Total Override*, are checked sequentially using the algorithms below.

Sanity Sanity rules out distinct rules that share an identical body but assign different labels, a configuration MSW cannot resolve by specificity and which has no consistent KLM counterpart. It is checked by pairwise comparison of rule bodies , and reporting a violation if there exists two rule with identical rule bodies but different labels.

Strict Global Exception Closure Strict Global Exception Closure is checked by pairwise comparison of rule bodies of opposite labels(Algorithm 3).

Algorithm 3 Strict Global Exception Closure

Input: Ruleset R
Output: Bool

for all $r_1 \in R$ **do**
  for all $r_2 \in R$ **do**
    if $r_1 = r_2$ **or** $r_1.label = r_2.label$ **then**
      continue
    end if
    $B_1 \leftarrow r_1.body$; $B_2 \leftarrow r_2.body$
    if compatible(B_1, B_2) **and** $B_1 \not\subset B_2$ **and** $B_2 \not\subset B_1$ **then**
      return False
    end if
  end for
end for
return True

No total Override The check in algorithm 4 verifies the No-Total-Override condition: every rule must have at least one complete feature pattern on which it is not defeated by a strictly more specific opposite-label rule. For a rule r, its own literals are fixed and the remaining features are free; only free features occurring in opposite-label rules can affect the outcome, so the search ranges over assignments to those alone. A rule fails the check exactly when every such assignment is dominated by a more specific opposite-label rule.

Algorithm 4 TotalOverride check

Input: Rule set $\mathcal{R}$
Output: True if every rule has a witnessing pattern; False otherwise

for all $r = (B_r, \ell_r) \in \mathcal{R}$ **do**
  fixed $\leftarrow$ the literals of B_r
  free $\leftarrow \{a : a \notin B_r,\ a$ occurs in some opposite-label rule$\}$
  if not HasWitness(r, *fixed*, *free*, $\mathcal{R}$) **then**
    return False
  end if
end for
return True

MSW and Explanations. The MSW side performs two jobs. First, it determines the label for a given instance, simultaneously collecting the applicable rules $A(F)$ and the inclusion-maximal applicable set $M(F)$, and marking, among applicable rules, those overridden by a strictly more specific applicable rule. The rules that survive, those not strictly overridden, form $M(F)$; the specificity comparisons recorded along the way form a chain of overrides. This chain is what makes the decision explainable: the same specificity ordering that selects the surviving rules also yields a trace of why each overridden rule was set aside.

For a rule set $\mathcal{R} = \{r_1, r_2, r_3, \dots\}$ and an instance F, a trace has the form shown in Figure 2.

```
Applicable rules: {r2, r6, r10}
Overrides:r10 applies
       r6 applies and is more specific than r10,
             so r10 is overridden
       r2 applies and is more specific than r6,
             so r6  is overridden
Inclusion-maximal rule: {r2}
Label: 1
```

Figure 2: Example MSW explanation trace.

The trace makes the decision fully auditable: it names every applicable rule, records the specificity comparison that overrode each one, and identifies the inclusion-maximal rules responsible for the label.

ECCRS to KLM translation. The mapping from an ECCRS rule set R to the defeasible knowledge base $\mathcal{K}(R)$ is defined in Section 3.3. Its implementation is a single-pass, purely syntactic transformation over the rule structure: each rule body, already stored as a set of signed literals, becomes the antecedent of a conditional, and the rule label selects the consequent literal (m or $\neg m$). No inference or search is performed, so the translation is linear in the size of the rule set, and it preserves exactly the information MSW consults, rule bodies and their labels. This is what allows the alignment result of Section 3.3 to compare the two sides on equal footing.

Lexicographic Closure implementation. Given the translated knowledge base $\mathcal{K}(R)$, the workbench answers classification queries using Lexicographic Closure (LC). The rank-based procedure underlying LC is described in Section 3; here we describe how it is realised in the implementation and how it is used to classify and explain instances.

Entailment via SAT. Each step of the LC procedure reduces to a propositional satisfiability test. For example, BaseRank (Algorithm 1) checks whether the antecedent α of a conditional is entailed by a subset E_i of the knowledge base, which reduces to testing whether $E_i \cup \{\alpha\}$ is unsatisfiable [12]. The implementation discharges these tests with the PicoSAT solver rather than a bespoke checker. Because the materialised knowledge base is already propositional, the reduction is direct: consistency of a candidate query against a level of the ranked knowledge base is encoded as a SAT instance and decided by the solver. Delegating to an established solver keeps the entailment core small and lets us rely on a well-tested decision procedure for the part of the pipeline most sensitive to correctness.

Classifying an instance and Explanation Traces. The algorithm 5 below shows how the instance is classified, and how explanation traces are generated in the process:

Algorithm 5 ClassifyInstanceWithLC

Input: KnowledgeBase $\mathcal{K}$, Instance F
Output: Classification for F

$q_m \leftarrow (F \mid\sim m)$
$q_{\neg m} \leftarrow (F \mid\sim \neg m)$
Trace("Candidate queries: q_m, $q_{\neg m}$")
entailM $\leftarrow$ LexicographicClosure($\mathcal{K}, q_m$)
entailNotM $\leftarrow$ LexicographicClosure($\mathcal{K}, q_{\neg m}$)
if *entailM* **then**
    Trace("Classification: $F \mid\sim m$ is entailed")
    return 1
else if *entailNotM* **then**
    Trace("Classification: $F \mid\sim \neg m$ is entailed")
    return 0
else
    Trace("Classification: abstain, neither query is entailed")
    return $\perp$
end if

For a complete feature pattern F, the workbench forms the two candidate queries $F \mid\sim m$ and $F \mid\sim \neg m$ and evaluates each under LC against $\mathcal{K}(R)$. It reports which of the defeasible implications are entailed, and reports Abstain if none. Figure 3 shows the corresponding KLM explanations of Figure 2 ECCRS'S explanations

```
Candidate Queries: F |~ m and F |~ ¬m
Classification: F |~ m is entailed
Label: LC entails label 1
```

Figure 3: Explanation Traces produces on the KLM side of the workbench

The final step is what makes the two sides of the workbench mutually supportive. The ECCRS-side trace (Figure 2) reports the override chain that resolves the applicable rules down to the inclusion-maximal set $M(F)$; the LC-side trace ties its entailed label back to those same rules. When the two agree, a prediction is justified twice over: structurally, by which rule survived MSW's specificity ordering, and semantically, by which label is entailed under a formally grounded defeasible-entailment relation.

These traces are also what make disagreement inspectable rather than opaque.

5.2 Experiment Setup

We evaluate on three datasets, Bank, Cancer and Mushroom, whose ECCRS rule sets were learned by the procedure of Madzime and Meyer [20] from UCI benchmarks [10] and are taken here as fixed input. Table 3 gives their sizes. Each instance is classified by MSW and by LC and assigned to exactly one of six outcomes: *Agree* (both decisive, same label), *Both abstain* (MSW abstains, LC entails neither m nor $\neg m$), *Disagree* (both decisive, different labels), *LC abstains* (MSW decisive, LC entails neither), *MSW abstains* (LC decisive, MSW abstains), and *Conflict* (LC entails both m and $\neg m$). The categories partition the instances, so their counts sum to N. We report alignment as (Agree + Both abstain)/N, since a shared abstention is a genuine agreement to withhold a label.

The workbench is evaluated only on instances with at least one applicable rule; this is done for two reasons: (1) The full feature space is too large, especially for datasets such as Bank and Mushroom, the feature space would be 2^{79} and 2^{106} respectively. (2) If the applicable rule set for a given Instance is empty, both sides trivially agree to abstain, so it contributes nothing to the alignment question.

All figures are over tested covered instances.

Table 3: Datasets used in the experiment

Dataset	N	Features	Rules
Bank	11303	79	21
Cancer	175	8	12
Mushroom	2031	106	13

Ethical considerations. The Cancer and Bank data are anonymised, publicly available UCI benchmarks [10] with no human-subject interaction, used here only to supply rule sets for verification; the safe-failure property studied assumes an abstention can be acted on responsibly rather than forced into a default decision.

Violations. To test what each assumption contributes, for each dataset we constructed three violating variants, each breaking one assumption. Each violation is induced by modifying a single rule in the base rule set: for *Sanity*, duplicating an existing rule and flipping its label; for *Closure*, adding a rule that is compatible to the existing rules but comparable to none; for *Total override*,one rule, $rz : (B_{rz}, l_{rz})$ from the ruleset , and a feature a^* not occurring in B_{rz} but present in the ruleset are chosen and two rules $rz' = (B_{rz} \wedge a^* = 1, 1-l_{rz})$ and $rz'' = (B_{rz} \wedge a^* = 0, 1-l_{rz})$ are produced from it, these two rules always override rz on every complete feature pattern to which rz is applicable.

This construction isolates the violation. For *Sanity* and *Total override* the added rules extend the body of an existing rule, so they apply only where that rule already applied and introduce no new covered instances; any deviation is attributable to the violation rather than to new coverage. The *Closure* rule is by definition comparable to no existing rule and may in principle cover new instances, but since it induces no relabelling on any dataset (Table 6), coverage effects do not confound its outcome.

As a concrete instance, consider rule 6 of the Bank rule set,

$$r_6 : a(10,2){=}0 \wedge a(16,3){=}1 \wedge a(4,1){=}0 \\ \wedge\ a(4,2){=}1 \wedge a(4,5){=}0 \wedge a(9,2){=}1 \Rightarrow 0.$$

We choose the feature $a(7,2)$, which does not occur in B_{r_6} but appears elsewhere in the rule set (in rules 2 and 11), and add the two opposite-label rules

$$r_{21} :\ B_{r_6} \wedge a(7,2){=}1 \Rightarrow 1; \qquad r_{22} :\ B_{r_6} \wedge a(7,2){=}0 \Rightarrow 1.$$

Every complete pattern fixes $a(7,2)$ to 0 or 1, so on any instance to which r_6 applies, exactly one of r_{21}, r_{22} also applies; each extends B_{r_6} by one literal and carries the opposite label, so it is strictly more specific than r_6 and overrides it. Rule r_6 is therefore defeated on every pattern it covers, violating No Total Override while introducing no coverage r_6 did not already have.

One caveat governs how the resulting counts should be read. The number of instances affected by a violation depends on how many test instances the modified rule decides, which is a property of the rule we chose and of the test set, not of the assumption itself. Violating a rule that decides many instances produces many divergences; violating a rule that decides few produces few. The magnitudes in Tables 5 and 6 are therefore illustrative of one chosen violation per assumption and are not comparable across assumptions or datasets as measures of severity. What is invariant, and what we report as the finding, is the *type* of divergence each violation produces, its failure signature, which does not depend on the rule chosen.

6 Results

On rule sets satisfying all three assumptions, every tested instance falls into *Agree* (Table 4); the other five categories (Both Abstain, Disagree, LC abstains, MSW abstains, Conflict) are empty for all datasets.

Table 4: Alignment of ECCRS and LC on rule sets that satisfies all assumptions

Dataset	N	Agree	Aligned (%)
Bank	11303	11303	100.00
Cancer	175	175	100.00
Mushroom	2031	2031	100.00
Overall	13509	13509	100.00

Table 5 reports the MSW–LC outcomes when each assumption is broken in turn. Breaking *Sanity* or *Total override* produces *Conflict* cases, in which LC entails both m and $\neg m$; breaking *Closure* produces no conflict, only abstention. In no case does any instance become *Disagree*.

Table 5: Failure signature of each violated assumption on the MSW–LC comparison. Counts are illustrative of one chosen violation, not severity measures. No violation produces a *Disagree*.

Dataset	Assumption	Agree	B.abst	M.abst	Conflict
Bank	Sanity	11032	0	0	271
	Closure	11200	0	103	0
	Total override	11289	0	0	14
Cancer	Sanity	92	0	0	83
	Closure	172	3	0	0
	Total override	98	0	0	77
Mushroom	Sanity	895	0	0	1136
	Closure	2023	8	0	0
	Total override	1987	0	0	44

Table 6 compares the clean ECCRS rule set against each violating variant. Wherever a violation reaches ECCRS's own decisions its direction is the same: breaking *Sanity* or *Closure* only ever causes abstention on instances the clean set had labelled, never a changed label, whereas breaking *Total override* changes labels (*Disagree*) without new abstentions. On Bank the *Sanity* violation leaves ECCRS untouched because the duplicated rule (rule 18) is never inclusion-maximal on any tested instance, yet LC still conflicts on 271 instances (Table 5) since it materialises the whole knowledge base; the verification layer detects a defect the classifier structurally cannot. As with the MSW–LC comparison, the number of affected instances varies with the chosen rule, but the direction of failure does not: on Mushroom, for example, *Sanity* moves 808 instances to abstention, *Closure* moves 8, and *Total override* relabels 44.

Table 6: Failure signature on the clean-versus-violating ECCRS comparison. *Disagree* counts changed labels; *Viol. abst.* counts instances the violating rule set leaves undecided that the clean set labeled. Under *Total override* the ECCRS classifier itself relabels, whereas the MSW–LC comparison only ever loses consistency.

Dataset	Assumption	Agree	Disagree	Viol. abst.
Bank	Sanity	11303	0	0
	Closure	11200	0	103
	Total override	11289	14	0
Cancer	Sanity	115	0	60
	Closure	172	0	3
	Total override	98	77	0
Mushroom	Sanity	1223	0	808
	Closure	2023	0	8
	Total override	1987	44	0

A worked divergence. We trace one instance from the *Sanity*-violating Cancer variant to show why a violation surfaces as *Conflict* rather than a wrong label. The violation duplicates rule 11, $a(2)=1 \Rightarrow 0$, as rule 12, $a(2)=1 \Rightarrow 1$: identical bodies, opposite labels. Instance 178 satisfies $a(2)=1$, so both rules apply.

On the ECCRS side, rules 11 and 12 are both applicable and neither is more specific than the other, so both remain in the inclusion-maximal set $M(F)$ with opposing labels. MSW therefore abstains, as it does whenever $M(F)$ carries mixed labels.

On the defeasible-entailment side, the translated knowledge base contains both $a(2) \mid\sim m$ (from rule 12) and $a(2) \mid\sim \neg m$ (from rule 11). For instance 178 Lexicographic Closure entails *both* $F \mid\sim m$ and $F \mid\sim \neg m$; the workbench records this as a *Conflict*.

The two sides decline to commit for the same underlying reason, a pair of identical-body, opposite-label rules, but express it differently: MSW abstains because no rule wins, while LC over-commits by entailing both labels. Crucially, neither side asserts the wrong label. The paired traces localise the inconsistency to rules 11 and 12, exactly the pair the violation introduced.

7 Discussion

Our first aim was to determine whether, under the alignment theorem's conditions, Lexicographic Closure agrees with MSW on the translated knowledge base. Both hold on the rule sets that satisfy the conditions; LC and MSW agreed on every tested instance,

and the paired traces resolved to the same label. The significance, however, is not the agreement alone but what the traces are and what they reveal: two independently computed justifications for the same decision, one structural (the rule(s) that survives MSW specificity overrides) and one semantic (the label entailed under a formal defeasible entailment relation). Because they provably coincide under the assumptions, ECCRS's cheap computation and human-readable trace can be argued to carry the same authority as the formal entailment it agrees with.

The value of the violation experiment is that it turns the Alignment theorem's assumptions from stated stipulations into observed necessities. Each assumption, when removed, produces a distinct and explicable failure, and the failure type reveals what the assumption was structurally guaranteeing. The number of instances affected varies and is not comparable across assumptions (Section 6); what is fixed , and what we read here, is the *kind* of failure and how it arises.

Two of the three assumptions turn out to fail in exactly the same way. Breaking Sanity adds a rule whose body is identical to an existing rule's but with the opposite label, placing both $B \mid\sim m$ and $B \mid\sim \neg m$ in the knowledge base; on any instance satisfying B the materialised base is classically inconsistent, and Lexicographic Closure entails both labels. Breaking No Total Override is less obviously similar but has the same root: the added rules make a rule's body inconsistent, because every completion of that body satisfies a more specific opposite-label rule, so again both m and $\neg m$ become derivable. In both cases the outcome is Conflict, and in both cases it is not incidental but the direct semantic image of an inconsistency the assumption exists to forbid. On the ECCRS side, No Total Override additionally shows up as a changed label (Disagree), since the more specific added rules also override the original for MSW. The same fault is thus, read once as a contradiction by the reasoner and once as a decision by the classifier.

Strict exception closure fails differently. Its violation adds a rule comparable to none of the existing rules, which creates no inconsistency; instead it leaves a case the specificity ranking cannot resolve, and MSW and LC abstain together rather than diverging. The effect on alignment is small and, on two of the three datasets, absent: Cancer and Mushroom stay at 100% because both sides abstain together, which our metric counts as agreement, while Bank drops slightly to 99.1 % because there the classifier abstains where the reasoner still decides. In no case does the violation produce conflict or a changed label. This sits outside what the alignment theorem promises, since the theorem only guarantees the undecided case when no rule applies, and here rules do apply; we therefore present the preserved alignment as an observed result rather than one the theorem predicts.

One subtlety qualifies the Closure reading. Its failure is abstention rather than conflict or relabeling on all three datasets, but the abstention is joint (both sides withhold) on Cancer and Mushroom and one-sided (MSW abstains, LC still decides) on Bank, where the added incomparable rule leaves MSW without an inclusion-maximal winner while LC can still rank the instance. The claim here is the negative one: a Closure violation never produces conflict or a changed label, only some loss of decisiveness and consistency.

Across every violation, no instance became a *Disagree* on the MSW-LC comparison. LC never entailed the opposite of MSW's label. This is not luck but a consequence of the translation. Each ECCRS rule maps to a conditional whose consequent is related to its own label(m for label 1; ¬m for label 0), therefore a malformed rule set can make both m and ¬m derivable (conflict) or neither (abstention), but nothing in the construction makes $\neg m$ derivable *in place of* m , and vice versa. The intended label is preserved even when the consistency and decisiveness is lost. This kind of failure mode, if made available to some sort of a classifier verification layer leads to a safe behavior,one that can withhold or flag contradiction. For example, an interpretable classifier intended for high stakes use, a system that abstains or signals inconsistency under malformed input is safer than one that silently misclassifies and the *Conflict* category makes the inconsistency visible rather than hidden. The guarantee is the verification layer's, not the classifier's: on the ECCRS side alone, breaking *Total override* does change predicted labels, so only checking against KLM entailment forces a malformed rule set to surface as a contradiction rather than a confident wrong answer.

The most instructive single finding is not any count but an asymmetry. Under *Total override* the same affected instances appear as *Conflict* on the MSW–LC comparison and as *Disagree* on the clean-versus-violating ECCRS comparison. The identical structural fault is read two ways: as a changed decision by the classifier, and as an inconsistency by the reasoner checking it. This is precisely the separation that gives the workbench its value. A relabeling that the classifier would report with full confidence is exactly what the translation converts into a visible contradiction.

8 Conclusions

This work asked whether the structural decisions of an ECCRS classifier, made by Most Specific Wins, coincide with the formal entailments of Lexicographic Closure on the translated knowledge base. On every tested rule set satisfying the alignment theorem's assumptions, the two agreed completely, and the paired explanation traces resolved to the same rule(s) and label. An ECCRS prediction is thus justified twice over, structurally and semantically, without any separate explanatory machinery.

When the assumptions are violated, the correspondence degrades in a bounded and safe way: the verification layer never entails the opposite of MSW's label, failing instead by withholding a label or by flagging a contradiction. Each assumption is characterised by the distinct failure it prevents, which gives the theorem's preconditions a concrete operational meaning rather than a purely formal one. Together these results place ECCRS on a firm footing within the KLM tradition of defeasible reasoning: its transparent, rule-level explanations carry the authority of a formally grounded entailment relation, and its guarantees are shown to hold exactly where the assumptions do. A direction that follows from here should concern the safe-failure property itself. Because a violated assumption surfaces as a detectable *Conflict* or joint abstention rather than a silent error, the translation can be used not only to certify a rule set but to audit one: run against a learned ECCRS rule set, it flags exactly the structural defects the assumptions forbid, turning the verification pipeline into a consistency checker for rule learners. That the failures are distinguishable by type further suggests which defects a learner most needs to prevent.

Declaration on the Use of Generative AI

During the preparation of this work, the author used Claude to receive critical feedback on argument, structure, and clarity, identify grammatical and formatting errors; paraphrasing and get suggestions on the organisation of the report . The author reviewed and edited all content and takes full responsibility for the final submission.

References

[1] Kumar Abhishek and Deeksha Kamath. 2022. Attribution-based XAI Methods in Computer Vision: A Review. *arXiv preprint arXiv:2211.14736* (2022).

[2] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. 2020. Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI. *Information Fusion* 58 (2020), 82–115. doi:10.1016/j.inffus.2019.12.012

[3] Mordechai Ben-Ari. 2012. *Mathematical Logic for Computer Science.* Springer London. https://doi.org/10.1007/978-1-4471-4129-7

[4] Alexander Bochman. 2008. Default logic generalized and simplified. *Annals of Mathematics and Artificial Intelligence* 53 (2008), 21–49. doi:10.1007/s10472-009-9109-x

[5] Katarina Britz, Giovanni Casini, Thomas Meyer, Kody Moodley, Uli Sattler, and Ivan Varzinczak. 2020. Principles of KLM-style Defeasible Description Logics. *ACM Transactions on Computational Logic* 22, 1, Article 1 (2020), 46 pages. doi:10.1145/3420258

[6] Lucas Carr, Nicholas Leisegang, Thomas Meyer, and Sebastian Rudolph. 2024. Non-monotonic Extensions to Formal Concept Analysis via Object Preferences. In *Artificial Intelligence Research: 5th Southern African Conference on AI Research (SACAIR 2024) (Communications in Computer and Information Science, Vol. 2326)*, Aurona Gerber, Jacomine Maritz, and Anban W. Pillay (Eds.). Springer, 476–492.

[7] Giovanni Casini, Thomas Meyer, and Ivan Varzinczak. 2019. *Taking Defeasible Entailment Beyond Rational Closure.* 182–197. doi:10.1007/978-3-030-19570-0_12

[8] Adnan Darwiche and Auguste Hirth. 2023. On the (complete) reasons behind decisions. *Journal of Logic, Language and Information* 32, 1 (2023), 63–88.

[9] Federica Di Stefano, Magdalena Ortiz, and Mantas Šimkus. 2022. Pointwise Circumscription in Description Logics. In *Proceedings of the 35th International Workshop on Description Logics (DL 2022) (CEUR Workshop Proceedings, Vol. 3263)*, Ofer Arieli, Martin Homola, Jean Christoph Jung, and Marie-Laure Mugnier (Eds.). CEUR-WS.org. https://ceur-ws.org/Vol-3263/paper-7.pdf

[10] Dheeru Dua and Casey Graff. 2019. UCI Machine Learning Repository. https://archive.ics.uci.edu/ml.

[11] Bishwamittra Ghosh, Dmitry Malioutov, and Kuldeep S. Meel. 2022. Efficient Learning of Interpretable Classification Rules. *Journal of Artificial Intelligence Research* 74 (2022), 1823–1863. doi:10.1613/jair.1.13482

[12] Adam Kaliski. 2020. An overview of KLM-style defeasible entailment. http://hdl.handle.net/11427/32743

[13] Christian Komo and Christoph Beierle. 2022. Nonmonotonic reasoning from conditional knowledge bases with system W. *Annals of Mathematics and Artificial Intelligence* 90, 1 (2022), 1–60. doi:10.1007/s10472-021-09777-9

[14] Sarit Kraus, Daniel Lehmann, and Menachem Magidor. 1990. Nonmonotonic reasoning, preferential models and cumulative logics. *Artificial Intelligence* 44, 1 (1990), 167–207. doi:10.1016/0004-3702(90)90101-5

[15] Himabindu Lakkaraju, Stephen H. Bach, and Jure Leskovec. 2016. Interpretable Decision Sets: A Joint Framework for Description and Prediction. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. ACM, 1675–1684. doi:10.1145/2939672.2939874

[16] Daniel Lehmann. 1995. Another perspective on default reasoning. *Annals of Mathematics and Artificial Intelligence* 15 (1995). doi:10.1007/BF01535841

[17] Daniel Lehmann and Menachem Magidor. 1992. What does a conditional knowledge base entail? *Artificial Intelligence* 55, 1 (1992), 1–60. doi:10.1016/0004-3702(92)90041-U

[18] Nicholas Leisegang, Thomas Meyer, and Sebastian Rudolph. 2024. Towards Propositional KLM-Style Defeasible Standpoint Logics. In *Artificial Intelligence Research: 5th Southern African Conference on AI Research (SACAIR 2024) (Communications in Computer and Information Science, Vol. 2326)*, Aurona Gerber, Jacomine Maritz, and Anban W. Pillay (Eds.). Springer, 459–475. doi:10.1007/978-3-031-78255-8_27

[19] Ruvarashe Madzime. 2026. Toward a Defeasible Semantics for Symbolic Classifiers. In *Proceedings of the Doctoral Consortium at KR 2026 (CEUR Workshop Proceedings, Vol. 4218)*. CEUR-WS.org, 14–21.

[20] Ruvarashe Madzime and Thomas Meyer. 2026. Toward Defeasible Machine Learning: When Learned Rule Sets Become Defeasible Theories. In *Proceedings of the 24th International Workshop on Nonmonotonic Reasoning (NMR 2026) (CEUR Workshop Proceedings, Vol. 4239)*. CEUR-WS.org. https://ceur-ws.org/Vol-4239/paper-Madzime-3.pdf

[21] Ruvarashe Shalom Madzime, Thomas Meyer, and Louise Leenen. 2025. An Override-Aware Classifier for Transparent AI. In *Proceedings of the Southern African Conference for Artificial Intelligence Research* (Cape Town, South Africa), Aurona Gerber and Anban Pillay (Eds.).

[22] Matthew Morris, Tala Ross, and Thomas Meyer. 2020. Algorithmic definitions for KLM-style defeasible disjunctive Datalog. *South African Computer Journal* 32, 2 (2020), 141–160. doi:10.18489/sacj.v32i2.846

[23] Alan G. Paredes Cetina, Kaouther Benguessoum, Raoni Lourenço, and Sylvain Kubler. 2026. Counterfactual Explainable AI (XAI) Method for Deep Learning-Based Multivariate Time Series Classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40.

[24] Michael Rapp, Johannes Fürnkranz, and Eyke Hüllermeier. 2024. On the efficient implementation of classification rule learning. *Advances in Data Analysis and Classification* 18, 4 (2024), 851–892. doi:10.1007/s11634-024-00579-7

[25] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. ACM, 1135–1144. doi:10.1145/2939672.2939778

[26] Ronald L. Rivest. 1987. Learning decision lists. *Machine Learning* 2, 3 (1987), 229–246. doi:10.1007/BF00058680

[27] Cynthia Rudin. 2019. Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. arXiv:1811.10154 [stat.ML] https://arxiv.org/abs/1811.10154

[28] Ricardo Vinuesa, Steven L. Brunton, and Gianmarco Mengaldo. 2026. Explainable AI: learning from the learners. *Nature Communications* 17 (2026), 7933. doi:10.1038/s41467-026-76359-w

[29] Shun Wang. 2023. *Defeasible Justification for the KLM Framework.* Master's thesis. University of Cape Town, Cape Town, South Africa. http://hdl.handle.net/11427/39922

Supplementary Information

A Rule Sets and Violation Constructions

This appendix lists, for each benchmark, the original exported ECCRS rule set, the rules added to induce each assumption violation, and the assumption validator's output on the resulting rule set. Together these allow the results of Section 6 to be reproduced.

A.1 Bank

Original rule set.

```
0:  a(15,9)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=0 & a(4,4)=1 & a(4,5)=0 -> 1
1:  a(15,9)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=0 & a(4,4)=0 & a(4,5)=0 -> 1
2:  a(10,2)=0 & a(12,2)=0 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 & a(7,2)=0 -> 1
3:  a(10,2)=1 & a(12,2)=0 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 & a(7,2)=0 -> 1
4:  a(12,1)=1 & a(14,3)=0 & a(15,1)=1 & a(16,3)=0 & a(3,5)=1 & a(4,5)=1 -> 1
5:  a(12,1)=1 & a(14,3)=0 & a(15,1)=1 & a(16,3)=0 & a(3,5)=0 & a(4,5)=1 -> 1
6:  a(10,2)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 & a(9,2)=1 -> 0
7:  a(12,1)=1 & a(15,9)=1 & a(16,3)=1 & a(4,1)=0 & a(4,2)=0 & a(4,5)=0 -> 1
8:  a(12,1)=0 & a(15,9)=1 & a(16,3)=1 & a(4,1)=0 & a(4,2)=0 & a(4,5)=0 -> 0
9:  a(10,2)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 & a(9,2)=0 -> 0
10: a(12,1)=1 & a(14,3)=0 & a(15,1)=1 & a(16,3)=0 & a(4,5)=1 -> 1
11: a(12,2)=0 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 & a(7,2)=1 -> 1
12: a(10,2)=1 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 -> 0
13: a(10,2)=1 & a(12,2)=1 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 -> 1
14: a(10,2)=0 & a(12,2)=1 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 -> 1
15: a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 -> 0
16: a(14,3)=0 & a(16,3)=1 & a(4,5)=1 -> 1
17: a(16,3)=1 & a(4,1)=1 & a(4,5)=0 -> 0
18: a(16,3)=1 & a(4,5)=0 -> 1
19: a(4,5)=0 -> 0
20: a(4,5)=1 -> 0
```

Sanity violation. Rule 21 duplicates the body of rule 18 with the opposite label, breaking Sanity.

```
21: a(16,3)=1 & a(4,5)=0 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Failed
  Reason: Rule 18 and Rule 21 have identical rule bodies but opposite labels

    2. Strict Global Exception Closure check: Failed
  Reason: Rule 18 and Rule 21 have opposite labels, compatible,
  neither a strict subset of the other
 3. No Total Override: Passed
```

Closure violation. Rule 21 is compatible with the existing rules but comparable to none, breaking strict exception closure.

```
21: a(10,2)=1 & a(9,2)=1 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Failed
  Reason: Rule 6 and Rule 21 have opposite labels, and are
  Compatible but neither is a strict subset of the other
 3. No Total Override: Passed
```

Total override violation. Rules 21 and 22 extend the body of rule 6 with $a(7,2)$=1 and $a(7,2)$=0 respectively, both with the opposite label, so that rule 6 is defeated on every pattern it covers.

```
21: a(10,2)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 & a(9,2)=1 & a(7,2)=1 -> 1
22: a(10,2)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 & a(9,2)=1 & a(7,2)=0 -> 1
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Passed
 3. No Total Override: Failed
  Reason:
  Rule 6 is completely overriden by Rule 21 and Rule 22
```

A.2 Mushroom

Original rule set.

```
0:  a(12,2)=0 & a(18,5)=0 & a(3,9)=0 & a(5,7)=1 & a(8)=1 -> 0
1:  a(1,4)=0 & a(4)=0 & a(5,5)=0 & a(5,7)=0 & a(5,8)=1 -> 1
2:  a(1,4)=1 & a(4)=0 & a(5,5)=0 & a(5,7)=0 & a(5,8)=1 -> 1
3:  a(12,2)=0 & a(18,5)=0 & a(3,9)=1 & a(5,7)=1 & a(8)=1 -> 0
```

```
4:  a(12,2)=0 & a(18,5)=0 & a(5,7)=1 & a(8)=0 -> 0
5:  a(4)=0 & a(5,5)=0 & a(5,7)=0 & a(5,8)=0 -> 0
6:  a(4)=0 & a(5,5)=0 & a(5,7)=0 & a(5,8)=1 -> 1
7:  a(12,2)=1 & a(16,3)=1 & a(18,5)=0 & a(5,7)=1 -> 0
8:  a(4)=0 & a(5,5)=1 & a(5,7)=0 -> 1
9:  a(12,2)=1 & a(18,5)=0 & a(5,7)=1 -> 1
10: a(4)=0 & a(5,7)=0 -> 0
11: a(18,5)=1 & a(5,7)=1 -> 1
12: a(5,7)=0 -> 1
```

Sanity violation. Rule 13 duplicates the body of rule 12 with the opposite label, breaking Sanity.

```
13: a(5,7)=0 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Failed
  Reason: Rule 12 and Rule 13 have identical rule bodies but opposite labels

    2. Strict Global Exception Closure check: Failed
  Reason: Rule 12 and Rule 13 have opposite labels, compatible,
  neither a strict subset of the other
 3. No Total Override: Passed
```

Closure violation. Rule 13 is compatible with the existing rules but comparable to none, breaking strict exception closure.

```
13: a(3,9)=1 & a(16,3)=1 & a(8)=0 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Failed
  Reason: opposite-label rule compatible with Rule 13 but
  neither is a strict subset of the other
 3. No Total Override: Passed
```

Total override violation. Rules 13 and 14 extend the body of rule 0 with $a(5,5)$=1 and $a(5,5)$=0 respectively, both with the opposite label, so that rule 0 is defeated on every pattern it covers.

```
13: a(12,2)=0 & a(18,5)=0 & a(3,9)=0 & a(5,7)=1 & a(8)=1 & a(5,5)=1 -> 1
14: a(12,2)=0 & a(18,5)=0 & a(3,9)=0 & a(5,7)=1 & a(8)=1 & a(5,5)=0 -> 1
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Passed
 3. No Total Override: Failed
  Reason: Rule 0 is completely overriden by Rule 13 and Rule 14
```

A.3 Cancer

Original rule set.

```
0:  a(2)=0 & a(4)=1 & a(7)=1 & a(8)=0 -> 1
1:  a(2)=0 & a(4)=1 & a(7)=0 & a(8)=0 -> 1
2:  a(1)=0 & a(2)=1 & a(4)=0 & a(5)=0 -> 1
3:  a(1)=1 & a(2)=1 & a(5)=0 & a(7)=1 -> 0
4:  a(1)=0 & a(2)=1 & a(4)=1 & a(5)=0 -> 1
5:  a(1)=1 & a(2)=1 & a(5)=0 & a(7)=0 -> 1
6:  a(2)=0 & a(4)=0 & a(8)=0 -> 1
7:  a(1)=1 & a(2)=1 & a(5)=0 -> 0
8:  a(2)=1 & a(5)=1 & a(7)=0 -> 1
9:  a(2)=1 & a(5)=0 -> 1
10: a(2)=0 & a(8)=1 -> 0
11: a(2)=1 -> 0
```

Sanity violation. Rule 12 duplicates the body of rule 11 with the opposite label, breaking Sanity.

```
12: a(2)=1 -> 1
```

Validator output:

```
Assumption Verification
 1. Sanity check: Failed
  Reason: Rule 11 and Rule 12 have identical rule bodies but opposite labels

    2. Strict Global Exception Closure check: Failed
  Reason: Rule 11 and Rule 12 have opposite labels, compatible,
  neither a strict subset of the other
 3. No Total Override: Passed
```

Closure violation. Rule 12 is compatible with the existing rules but comparable to none, breaking strict exception closure.

```
12: a(4)=1 & a(5)=1 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Failed
  Reason: Rule 8 and Rule 12 have opposite labels, compatible,
  neither a strict subset of the other
 3. No Total Override: Passed
```

Total override violation. Rules 12 and 13 extend the body of rule 6 with $a(7)$=1 and $a(7)$=0 respectively, both with the opposite label, so that rule 6 is defeated on every pattern it covers.

```
12: a(2)=0 & a(4)=0 & a(8)=0 & a(7)=1 -> 0
13: a(2)=0 & a(4)=0 & a(8)=0 & a(7)=0 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Passed
 3. No Total Override: Failed
  Reason: Rule 6 is completely overriden by Rule 12 and Rule 13
```

B Full Agreement Tables

For each violated assumption, the tables below give the complete six-outcome breakdown of the MSW–LC comparison and the ECCRS clean-versus-violating comparison. Each row is the dataset whose rule set was violated, taken from its own run; the two unmodified control datasets in each run remain fully aligned and are omitted.

B.1 MSW–LC comparison

Table 7: MSW–LC outcomes, Sanity violation.

Dataset	*N*	Agree	Both abst.	Disagree	LC abst.	MSW abst.	Conflict
Bank	11303	11032	0	0	0	0	271
Cancer	175	92	0	0	0	0	83
Mushroom	2031	895	0	0	0	0	1136

Table 8: MSW–LC outcomes, Closure violation.

Dataset	*N*	Agree	Both abst.	Disagree	LC abst.	MSW abst.	Conflict
Bank	11303	11200	0	0	0	103	0
Cancer	175	172	3	0	0	0	0
Mushroom	2031	2023	8	0	0	0	0

B.2 ECCRS clean-versus-violating comparison

Table 9: MSW–LC outcomes, Total override violation.

Dataset	N	Agree	Both abst.	Disagree	LC abst.	MSW abst.	Conflict
Bank	11303	11289	0	0	0	0	14
Cancer	175	98	0	0	0	0	77
Mushroom	2031	1987	0	0	0	0	44

Table 10: Clean versus violating ECCRS, Sanity violation.

Dataset	N	Agree	Disagree	Both abst.	Viol. abst.
Bank	11303	11303	0	0	0
Cancer	175	115	0	0	60
Mushroom	2031	1223	0	0	808

Table 11: Clean versus violating ECCRS, Closure violation.

Dataset	N	Agree	Disagree	Both abst.	Viol. abst.
Bank	11303	11200	0	0	103
Cancer	175	172	0	0	3
Mushroom	2031	2023	0	0	8

Table 12: Clean versus violating ECCRS, Total override violation.

Dataset	N	Agree	Disagree	Both abst.	Viol. abst.
Bank	11303	11289	14	0	0
Cancer	175	98	77	0	0
Mushroom	2031	1987	44	0	0

C Witness Search

Algorithm 7 is the witness search invoked by the No-Total-Override check. It enumerates completions of the free features and returns true as soon as one leaves r un-overridden.

References

[1] Kumar Abhishek and Deeksha Kamath. 2022. Attribution-based XAI Methods in Computer Vision: A Review. *arXiv preprint arXiv:2211.14736* (2022).

Algorithm 6 HasWitness: does r survive on some complete pattern?

Input: Rule $r = (B_r, \ell_r)$, fixed literals, free features, rule set $\mathcal{R}$
Output: True if some completion leaves r un-overridden

```
for all complete assignments F extending fixed over free do
    if no rule (C, 1 − ℓ_r) ∈ R with B_r ⊂ C is applicable to F then
        return True                                   ▷ F witnesses that r is not overridden
    end if
end for
return False
```


[2] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. 2020. Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI. *Information Fusion* 58 (2020), 82–115. doi:10.1016/j.inffus.2019.12.012

[3] Mordechai Ben-Ari. 2012. *Mathematical Logic for Computer Science.* Springer London. https://doi.org/10.1007/978-1-4471-4129-7

[4] Alexander Bochman. 2008. Default logic generalized and simplified. *Annals of Mathematics and Artificial Intelligence* 53 (2008), 21–49. doi:10.1007/s10472-009-9109-x

[5] Katarina Britz, Giovanni Casini, Thomas Meyer, Kody Moodley, Uli Sattler, and Ivan Varzinczak. 2020. Principles of KLM-style Defeasible Description Logics. *ACM Transactions on Computational Logic* 22, 1, Article 1 (2020), 46 pages. doi:10.1145/3420258

[6] Lucas Carr, Nicholas Leisegang, Thomas Meyer, and Sebastian Rudolph. 2024. Non-monotonic Extensions to Formal Concept Analysis via Object Preferences. In *Artificial Intelligence Research: 5th Southern African Conference on AI Research (SACAIR 2024) (Communications in Computer and Information Science, Vol. 2326)*, Aurona Gerber, Jacomine Maritz, and Anban W. Pillay (Eds.). Springer, 476–492.

[7] Giovanni Casini, Thomas Meyer, and Ivan Varzinczak. 2019. *Taking Defeasible Entailment Beyond Rational Closure.* 182–197. doi:10.1007/978-3-030-19570-0_12

[8] Adnan Darwiche and Auguste Hirth. 2023. On the (complete) reasons behind decisions. *Journal of Logic, Language and Information* 32, 1 (2023), 63–88.

[9] Federica Di Stefano, Magdalena Ortiz, and Mantas Šimkus. 2022. Pointwise Circumscription in Description Logics. In *Proceedings of the 35th International Workshop on Description Logics (DL 2022) (CEUR Workshop Proceedings, Vol. 3263)*, Ofer Arieli, Martin Homola, Jean Christoph Jung, and Marie-Laure Mugnier (Eds.). CEUR-WS.org. https://ceur-ws.org/Vol-3263/paper-7.pdf

[10] Dheeru Dua and Casey Graff. 2019. UCI Machine Learning Repository. https://archive.ics.uci.edu/ml.

[11] Bishwamittra Ghosh, Dmitry Malioutov, and Kuldeep S. Meel. 2022. Efficient Learning of Interpretable Classification Rules. *Journal of Artificial Intelligence Research* 74 (2022), 1823–1863. doi:10.1613/jair.1.13482

[12] Adam Kaliski. 2020. An overview of KLM-style defeasible entailment. http://hdl.handle.net/11427/32743

[13] Christian Komo and Christoph Beierle. 2022. Nonmonotonic reasoning from conditional knowledge bases with system W. *Annals of Mathematics and Artificial Intelligence* 90, 1 (2022), 1–60. doi:10.1007/s10472-021-09777-9

[14] Sarit Kraus, Daniel Lehmann, and Menachem Magidor. 1990. Nonmonotonic reasoning, preferential models and cumulative logics. *Artificial Intelligence* 44, 1 (1990), 167–207. doi:10.1016/0004-3702(90)90101-5

[15] Himabindu Lakkaraju, Stephen H. Bach, and Jure Leskovec. 2016. Interpretable Decision Sets: A Joint Framework for Description and Prediction. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. ACM, 1675–1684. doi:10.1145/2939672.2939874

[16] Daniel Lehmann. 1995. Another perspective on default reasoning. *Annals of Mathematics and Artificial Intelligence* 15 (1995). doi:10.1007/BF01535841

[17] Daniel Lehmann and Menachem Magidor. 1992. What does a conditional knowledge base entail? *Artificial Intelligence* 55, 1 (1992), 1–60. doi:10.1016/0004-3702(92)90041-U

[18] Nicholas Leisegang, Thomas Meyer, and Sebastian Rudolph. 2024. Towards Propositional KLM-Style Defeasible Standpoint Logics. In *Artificial Intelligence Research: 5th Southern African Conference on AI Research (SACAIR 2024) (Communications in Computer and Information Science, Vol. 2326)*, Aurona Gerber, Jacomine Maritz, and Anban W. Pillay (Eds.). Springer, 459–475. doi:10.1007/978-3-031-78255-8_27

[19] Ruvarashe Madzime. 2026. Toward a Defeasible Semantics for Symbolic Classifiers. In *Proceedings of the Doctoral Consortium at KR 2026 (CEUR Workshop Proceedings, Vol. 4218)*. CEUR-WS.org, 14–21.

[20] Ruvarashe Madzime and Thomas Meyer. 2026. Toward Defeasible Machine Learning: When Learned Rule Sets Become Defeasible Theories. In *Proceedings of the 24th International Workshop on Nonmonotonic Reasoning (NMR 2026) (CEUR Workshop Proceedings, Vol. 4239)*. CEUR-WS.org. https://ceur-ws.org/Vol-4239/paper-Madzime-3.pdf

[21] Ruvarashe Shalom Madzime, Thomas Meyer, and Louise Leenen. 2025. An Override-Aware Classifier for Transparent AI. In *Proceedings of the Southern African Conference for Artificial Intelligence Research* (Cape Town, South Africa), Aurona Gerber and Anban Pillay (Eds.).

[22] Matthew Morris, Tala Ross, and Thomas Meyer. 2020. Algorithmic definitions for KLM-style defeasible disjunctive Datalog. *South African Computer Journal* 32, 2 (2020), 141–160. doi:10.18489/sacj.v32i2.846

[23] Alan G. Paredes Cetina, Kaouther Benguessoum, Raoni Lourenço, and Sylvain Kubler. 2026. Counterfactual Explainable AI (XAI) Method for Deep Learning-Based Multivariate Time Series Classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40.

[24] Michael Rapp, Johannes Fürnkranz, and Eyke Hüllermeier. 2024. On the efficient implementation of classification rule learning. *Advances in Data Analysis and Classification* 18, 4 (2024), 851–892. doi:10.1007/s11634-024-00579-7

[25] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. ACM, 1135–1144. doi:10.1145/2939672.2939778

[26] Ronald L. Rivest. 1987. Learning decision lists. *Machine Learning* 2, 3 (1987), 229–246. doi:10.1007/BF00058680

[27] Cynthia Rudin. 2019. Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. arXiv:1811.10154 [stat.ML] https://arxiv.org/abs/1811.10154

[28] Ricardo Vinuesa, Steven L. Brunton, and Gianmarco Mengaldo. 2026. Explainable AI: learning from the learners. *Nature Communications* 17 (2026), 7933. doi:10.1038/s41467-026-76359-w

[29] Shun Wang. 2023. *Defeasible Justification for the KLM Framework.* Master's thesis. University of Cape Town, Cape Town, South Africa. http://hdl.handle.net/11427/39922

Supplementary Information

A Rule Sets and Violation Constructions

This appendix lists, for each benchmark, the original exported ECCRS rule set, the rules added to induce each assumption violation, and the assumption validator's output on the resulting rule set. Together these allow the results of Section 6 to be reproduced.

A.1 Bank

Original rule set.

```
0:  a(15,9)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=0 & a(4,4)=1 & a(4,5)=0 -> 1
1:  a(15,9)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=0 & a(4,4)=0 & a(4,5)=0 -> 1
2:  a(10,2)=0 & a(12,2)=0 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 & a(7,2)=0 -> 1
3:  a(10,2)=1 & a(12,2)=0 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 & a(7,2)=0 -> 1
4:  a(12,1)=1 & a(14,3)=0 & a(15,1)=1 & a(16,3)=0 & a(3,5)=1 & a(4,5)=1 -> 1
5:  a(12,1)=1 & a(14,3)=0 & a(15,1)=1 & a(16,3)=0 & a(3,5)=0 & a(4,5)=1 -> 1
6:  a(10,2)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 & a(9,2)=1 -> 0
7:  a(12,1)=1 & a(15,9)=1 & a(16,3)=1 & a(4,1)=0 & a(4,2)=0 & a(4,5)=0 -> 1
8:  a(12,1)=0 & a(15,9)=1 & a(16,3)=1 & a(4,1)=0 & a(4,2)=0 & a(4,5)=0 -> 0
9:  a(10,2)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 & a(9,2)=0 -> 0
10: a(12,1)=1 & a(14,3)=0 & a(15,1)=1 & a(16,3)=0 & a(4,5)=1 -> 1
11: a(12,2)=0 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 & a(7,2)=1 -> 1
12: a(10,2)=1 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 -> 0
13: a(10,2)=1 & a(12,2)=1 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 -> 1
14: a(10,2)=0 & a(12,2)=1 & a(14,3)=0 & a(16,3)=1 & a(4,5)=1 -> 1
15: a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 -> 0
16: a(14,3)=0 & a(16,3)=1 & a(4,5)=1 -> 1
17: a(16,3)=1 & a(4,1)=1 & a(4,5)=0 -> 0
18: a(16,3)=1 & a(4,5)=0 -> 1
19: a(4,5)=0 -> 0
20: a(4,5)=1 -> 0
```

Sanity violation. Rule 21 duplicates the body of rule 18 with the opposite label, breaking Sanity.

```
21: a(16,3)=1 & a(4,5)=0 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Failed
  Reason: Rule 18 and Rule 21 have identical rule bodies but opposite labels

    2. Strict Global Exception Closure check: Failed
  Reason: Rule 18 and Rule 21 have opposite labels, compatible,
  neither a strict subset of the other
 3. No Total Override: Passed
```

Closure violation. Rule 21 is compatible with the existing rules but comparable to none, breaking strict exception closure.

```
21: a(10,2)=1 & a(9,2)=1 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Failed
  Reason: Rule 6 and Rule 21 have opposite labels, and are
  Compatible but neither is a strict subset of the other
 3. No Total Override: Passed
```

Total override violation. Rules 21 and 22 extend the body of rule 6 with $a(7,2)$=1 and $a(7,2)$=0 respectively, both with the opposite label, so that rule 6 is defeated on every pattern it covers.

```
21: a(10,2)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 & a(9,2)=1 & a(7,2)=1 -> 1
22: a(10,2)=0 & a(16,3)=1 & a(4,1)=0 & a(4,2)=1 & a(4,5)=0 & a(9,2)=1 & a(7,2)=0 -> 1
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Passed
 3. No Total Override: Failed
  Reason:
  Rule 6 is completely overriden by Rule 21 and Rule 22
```

A.2 Mushroom

Original rule set.

```
0:  a(12,2)=0 & a(18,5)=0 & a(3,9)=0 & a(5,7)=1 & a(8)=1 -> 0
1:  a(1,4)=0 & a(4)=0 & a(5,5)=0 & a(5,7)=0 & a(5,8)=1 -> 1
2:  a(1,4)=1 & a(4)=0 & a(5,5)=0 & a(5,7)=0 & a(5,8)=1 -> 1
3:  a(12,2)=0 & a(18,5)=0 & a(3,9)=1 & a(5,7)=1 & a(8)=1 -> 0
```

```
4:  a(12,2)=0 & a(18,5)=0 & a(5,7)=1 & a(8)=0 -> 0
5:  a(4)=0 & a(5,5)=0 & a(5,7)=0 & a(5,8)=0 -> 0
6:  a(4)=0 & a(5,5)=0 & a(5,7)=0 & a(5,8)=1 -> 1
7:  a(12,2)=1 & a(16,3)=1 & a(18,5)=0 & a(5,7)=1 -> 0
8:  a(4)=0 & a(5,5)=1 & a(5,7)=0 -> 1
9:  a(12,2)=1 & a(18,5)=0 & a(5,7)=1 -> 1
10: a(4)=0 & a(5,7)=0 -> 0
11: a(18,5)=1 & a(5,7)=1 -> 1
12: a(5,7)=0 -> 1
```

Sanity violation. Rule 13 duplicates the body of rule 12 with the opposite label, breaking Sanity.

```
13: a(5,7)=0 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Failed
  Reason: Rule 12 and Rule 13 have identical rule bodies but opposite labels

    2. Strict Global Exception Closure check: Failed
  Reason: Rule 12 and Rule 13 have opposite labels, compatible,
  neither a strict subset of the other
 3. No Total Override: Passed
```

Closure violation. Rule 13 is compatible with the existing rules but comparable to none, breaking strict exception closure.

```
13: a(3,9)=1 & a(16,3)=1 & a(8)=0 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Failed
  Reason: opposite-label rule compatible with Rule 13 but
  neither is a strict subset of the other
 3. No Total Override: Passed
```

Total override violation. Rules 13 and 14 extend the body of rule 0 with $a(5,5)$=1 and $a(5,5)$=0 respectively, both with the opposite label, so that rule 0 is defeated on every pattern it covers.

```
13: a(12,2)=0 & a(18,5)=0 & a(3,9)=0 & a(5,7)=1 & a(8)=1 & a(5,5)=1 -> 1
14: a(12,2)=0 & a(18,5)=0 & a(3,9)=0 & a(5,7)=1 & a(8)=1 & a(5,5)=0 -> 1
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Passed
 3. No Total Override: Failed
  Reason: Rule 0 is completely overriden by Rule 13 and Rule 14
```

A.3 Cancer

Original rule set.

```
0:  a(2)=0 & a(4)=1 & a(7)=1 & a(8)=0 -> 1
1:  a(2)=0 & a(4)=1 & a(7)=0 & a(8)=0 -> 1
2:  a(1)=0 & a(2)=1 & a(4)=0 & a(5)=0 -> 1
3:  a(1)=1 & a(2)=1 & a(5)=0 & a(7)=1 -> 0
4:  a(1)=0 & a(2)=1 & a(4)=1 & a(5)=0 -> 1
5:  a(1)=1 & a(2)=1 & a(5)=0 & a(7)=0 -> 1
6:  a(2)=0 & a(4)=0 & a(8)=0 -> 1
7:  a(1)=1 & a(2)=1 & a(5)=0 -> 0
8:  a(2)=1 & a(5)=1 & a(7)=0 -> 1
9:  a(2)=1 & a(5)=0 -> 1
10: a(2)=0 & a(8)=1 -> 0
11: a(2)=1 -> 0
```

Sanity violation. Rule 12 duplicates the body of rule 11 with the opposite label, breaking Sanity.

```
12: a(2)=1 -> 1
```

Validator output:

```
Assumption Verification
 1. Sanity check: Failed
  Reason: Rule 11 and Rule 12 have identical rule bodies but opposite labels

    2. Strict Global Exception Closure check: Failed
  Reason: Rule 11 and Rule 12 have opposite labels, compatible,
  neither a strict subset of the other
 3. No Total Override: Passed
```

Closure violation. Rule 12 is compatible with the existing rules but comparable to none, breaking strict exception closure.

```
12: a(4)=1 & a(5)=1 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Failed
  Reason: Rule 8 and Rule 12 have opposite labels, compatible,
  neither a strict subset of the other
 3. No Total Override: Passed
```

Total override violation. Rules 12 and 13 extend the body of rule 6 with $a(7)$=1 and $a(7)$=0 respectively, both with the opposite label, so that rule 6 is defeated on every pattern it covers.

```
12: a(2)=0 & a(4)=0 & a(8)=0 & a(7)=1 -> 0
13: a(2)=0 & a(4)=0 & a(8)=0 & a(7)=0 -> 0
```

Validator output:

```
Assumption Verification
 1. Sanity check: Passed
    2. Strict Global Exception Closure check: Passed
 3. No Total Override: Failed
  Reason: Rule 6 is completely overriden by Rule 12 and Rule 13
```

B Full Agreement Tables

For each violated assumption, the tables below give the complete six-outcome breakdown of the MSW–LC comparison and the ECCRS clean-versus-violating comparison. Each row is the dataset whose rule set was violated, taken from its own run; the two unmodified control datasets in each run remain fully aligned and are omitted.

B.1 MSW–LC comparison

Table 13: MSW–LC outcomes, Sanity violation.

Dataset	*N*	Agree	Both abst.	Disagree	LC abst.	MSW abst.	Conflict
Bank	11303	11032	0	0	0	0	271
Cancer	175	92	0	0	0	0	83
Mushroom	2031	895	0	0	0	0	1136

Table 14: MSW–LC outcomes, Closure violation.

Dataset	*N*	Agree	Both abst.	Disagree	LC abst.	MSW abst.	Conflict
Bank	11303	11200	0	0	0	103	0
Cancer	175	172	3	0	0	0	0
Mushroom	2031	2023	8	0	0	0	0

B.2 ECCRS clean-versus-violating comparison

Table 15: MSW–LC outcomes, Total override violation.

Dataset	N	Agree	Both abst.	Disagree	LC abst.	MSW abst.	Conflict
Bank	11303	11289	0	0	0	0	14
Cancer	175	98	0	0	0	0	77
Mushroom	2031	1987	0	0	0	0	44

Table 16: Clean versus violating ECCRS, Sanity violation.

Dataset	N	Agree	Disagree	Both abst.	Viol. abst.
Bank	11303	11303	0	0	0
Cancer	175	115	0	0	60
Mushroom	2031	1223	0	0	808

Table 17: Clean versus violating ECCRS, Closure violation.

Dataset	N	Agree	Disagree	Both abst.	Viol. abst.
Bank	11303	11200	0	0	103
Cancer	175	172	0	0	3
Mushroom	2031	2023	0	0	8

Table 18: Clean versus violating ECCRS, Total override violation.

Dataset	N	Agree	Disagree	Both abst.	Viol. abst.
Bank	11303	11289	14	0	0
Cancer	175	98	77	0	0
Mushroom	2031	1987	44	0	0

C Witness Search

Algorithm 7 is the witness search invoked by the No-Total-Override check. It enumerates completions of the free features and returns true as soon as one leaves r un-overridden.

Algorithm 7 HasWitness: does r survive on some complete pattern?

Input: Rule $r = (B_r, \ell_r)$, fixed literals, free features, rule set $\mathcal{R}$
Output: TRUE if some completion leaves r un-overridden

for all complete assignments F extending *fixed* over *free* **do**
  if no rule $(C, 1 - \ell_r) \in \mathcal{R}$ with $B_r \subset C$ is applicable to F **then**
    return TRUE ▷ F witnesses that r is not overridden
  end if
end for
return FALSE